RUNBOOK — Fly GACA iOS apps (native family + store presence)

SECTION 06-operations-it TYPE runbook STATUS active OWNER Founder UPDATED 2026-08-05

> **Current track (2026-08).** The shipping iOS strategy is the **native six-app family** — one offline SwiftUI app per study module (PPL, ELPT, AIP, CPL, IR, ATPL), paid up-front, built from the shared `ay2m/FlyGACA` repo (FlyGACAKit + per-app content snapshots synced from the web monorepo). The single-app **Capacitor + Pro subscription** wrapper this runbook originally documented is **superseded for launch** and kept below as the reference for a future subscription phase.

0. The native six-app family — release path

What ships: six independent App Store apps (bundle ids `com.flygaca.<module>`), identical offline feature set (study mode, quizzing, flashcards with spaced repetition, mock tests, a timed scored exam simulation) — **no accounts, no IAP, no network in v1**; content is bundled at build time. Wave 1 = PPL · ELPT · AIP; Wave 2 = CPL · IR · ATPL (bank content in review). The operational detail lives with the code — this section only points:

1. **Content sync** — `scripts/sync-content.sh` in `ay2m/FlyGACA`; the monorepo stays the corpus source of truth (last full sync 2026-08-05).
2. **Build / test / CI** — the family repo's `docs/RUNBOOK-ios-release.md` and `.github/workflows/ios.yml`: Swift tests, XcodeGen validation, a six-app build matrix, and a Wave 1 TestFlight lane that activates once signing secrets exist.
3. **Apple portal (human, one-time)** — `docs/RUNBOOK-ios-signing.md` + `docs/RUNBOOK-ios-signing-CHECKLIST.md`: App Group `group.com.flygaca.study`; App IDs with **Sign in with Apple enabled and grouped under** `com.flygaca.ppl` (the shared entitlements already declare it — the first signed build fails provisioning until this is done); distribution certificate; `FlyGACA <APP> AppStore` provisioning profiles; paid App Store Connect records; the ASC API key; then the ten GitHub secrets (`scripts/native/set-signing-secrets.sh`).
4. **Firebase (for the later online phase)** — `docs/RUNBOOK-ios-firebase.md`: `npm run firebase:register` plus the manual Sign in with Apple provider and APNs key. Harmless to do early; not required for the offline v1.
5. **Store listings** — the six `FlyGACA/<MODULE>` metadata repos: EN + AR copy and screenshots, each gated by a per-repo CI check (limits + locale parity). Listing copy describes the offline, English-language app truthfully.

Apple review checklist — native offline v1

- **No IAP and no external purchase links** — the apps are paid up-front, so guideline 3.1.1 does not apply.
- **No account system in v1** — Sign in with Apple (4.8) and in-app account/data deletion (5.1.1) become requirements only when the online phase (PlatformLive) ships sign-in. The *capability* on the App IDs is still required for signing, because the shared entitlements declare it.
- **Privacy nutrition labels:** "Data Not Collected" (offline; no analytics, no tracking). Revisit the labels before any telemetry is ever added.
- **The "not for operational use" disclaimer stays prominent** in every app and listing — educational use only; GACA is always the authority.
- **Guideline 4.3(b) (spam/app farms)** — six sibling apps from one shell: keep the differentiation dossier per the family repo's `SEO-PLAN.md` §3.1 ready before submission.

Superseded track — single Capacitor app + Pro subscription (future option)

> Everything below documents the **pre-2026-08 plan**: one `com.flygaca.app` Capacitor shell around the web PWA, monetised with RevenueCat / Apple IAP. The monorepo code it references still exists, and this remains the reference if a subscription wrapper is revived — but it is **not** the path to the current App Store launch.

The iOS app wraps the existing web PWA in a Capacitor shell and sells **Fly GACA Pro** via Apple In-App Purchase. It reuses the entire backend: a purchase grants `users/{uid}.entitlement` (written only by a Cloud Function), and the app reads it through the same `store.js` / `entitlements.js` path the web uses.

What's already in the repo (built and web-safe):

- `capacitor.config.ts` — appId `com.flygaca.app`, `webDir: www`.
- `npm run build:ios` (`scripts/build-ios-web.js`) — assembles `www/`: bundles GACAR `parts/`, VFR `charts/`, indexes, tools, study, guides (~26 MB); excludes the FAA/foreign handbooks, ebooks and the 19 MB search index (these stream + cache on device). Injects `assets/js/native-bridge.js`.
- `assets/js/native-bridge.js` — native-only shim: routes the paywall CTA to the StoreKit sheet, RevenueCat purchase/restore, and a fetch-cache for the streamed assets. No-op on web.
- `assets/js/auth.js` — native Apple/Google sign-in branch (web still uses popup).
- `assets/js/gate.js` — paywall is LIVE only inside the app (`GATING_LIVE` is true only when `Capacitor.isNativePlatform()`); the web stays open.
- `functions/revenuecatWebhook.js` — RevenueCat → `users/{uid}.entitlement`.
- `firebase.json` — CORS on `/assets/data/**` so the app can stream heavy assets.

Everything below needs a **Mac with Xcode + CocoaPods** and the **Apple Developer** and **RevenueCat** accounts — it cannot be done in CI/Linux.

---

1. Generate the native project (Mac)

```
npm install --legacy-peer-deps        # .npmrc already sets this
npm run build:ios                     # writes www/
npx cap add ios                       # generates ios/ (Xcode project + Pods)
npx cap sync ios                      # or: npm run cap:sync
npx cap open ios                      # opens Xcode
```

In Xcode: set the Team/signing, bundle id `com.flygaca.app`, and enable the **In-App Purchase** and **Sign in with Apple** capabilities (and **Push Notifications** when Phase 2 lands).

2. Firebase native auth

- Add an **iOS app** to the `flygaca-firebase` Firebase project; download `GoogleService-Info.plist` into `ios/App/App/`.
- Enable **Apple** and **Google** providers in Firebase Auth (already enabled for web). Add the iOS bundle id. For Google sign-in, add the reversed client id URL scheme to the iOS target.
- Confirm `@capacitor-firebase/authentication` method/result shapes against the installed version (v7.x): `signInWithApple()`, `signInWithGoogle()` → `result.credential.{idToken,nonce,accessToken}`. The branch in `auth.js` exchanges these into the JS SDK via `signInWithCredential`.

3. App Store Connect — subscription products

Create an **auto-renewable subscription group** "Fly GACA Pro" with:

- `pro_monthly` — SAR 59 / month.
- `pro_annual` — SAR 349 / year, with a **7-day free trial** (intro offer). (Product ids containing `annual` / `year` map to `source: 'annual'` in the webhook; edit `periodOf()` in `functions/revenuecatWebhook.js` if you choose other ids.) Add the required localized metadata, review screenshot, and the **subscription terms / privacy** links.

4. RevenueCat

- Create a project, add the App Store app + the App-Specific Shared Secret.
- Create an **Offering** with `$rc_monthly` and `$rc_annual` packages mapped to the two products, and an **Entitlement** named `pro` attached to both.
- Copy the **iOS public SDK key** → set it for the app. Either inline in `native-bridge.js` (`REVENUECAT_IOS_KEY`) or expose `window.FG_REVENUECAT_IOS_KEY` before that script loads. It is

a publishable key, safe to ship.

- **Webhook**: point it at the deployed function URL (`https://me-central1-flygaca-firebase.cloudfunctions.net/revenuecatWebhook` or the Cloud Run URL) and set the **Authorization** header to a strong secret.
- The app calls `Purchases.logIn(firebaseUid)` after sign-in (via `FGNative.setUser`) so `app_user_id` equals the Firebase uid.

5. Deploy the webhook

```
firebase functions:secrets:set REVENUECAT_WEBHOOK_AUTH   # same value as the RC header
cd functions && npm ci && cd ..
firebase deploy --only functions:revenuecatWebhook,hosting
```

(The `hosting` deploy ships the new `/assets/data/**` CORS header.)

6. Verify the purchase loop (StoreKit sandbox)

1. Sign in (Apple/Google) on a device/simulator.
2. Hit a gated feature (e.g. a 4th flight tool, or Study) → the paywall shows **Go Pro** → buy with a sandbox account.
3. RevenueCat shows the `pro` entitlement active → its webhook writes `users/{uid}.entitlement` (check Firestore) → the app reloads and the gate lifts.
4. Sign in to the **web** with the same account → Pro shows there too.
5. Test **Restore Purchases** and the annual intro trial.
6. Offline: open a GACAR Part and a chart with networking off (bundled → work); open an FAA handbook online once, then offline (streamed → cached).

Apple review checklist (Capacitor + IAP track)

IAP-only for Pro (3.1.1) · native Sign in with Apple (4.8) · in-app account + data deletion (already in `settings.js`, 5.1.1) · Restore Purchases · privacy nutrition labels · keep the "not for operational use" disclaimer prominent.

Phase 2 — expiry reminders + Prep Packs (in the repo; needs device wiring)

Push expiry reminders

Code already present: `functions/reminders.js` (daily `expiryReminders` scheduled job at 06:00 Asia/Riyadh — scans `users`, pushes when `medicalExpiry` / `lastFlightReview` +24mo cross a threshold), `store.js savePushToken`, `native-bridge.js registerPush`, and the dashboard hook that registers on login. To activate:

1. Apple Developer → create an **APNs Auth Key (.p8)**; upload it to Firebase Project Settings → Cloud Messaging (iOS).
2. Xcode → add the **Push Notifications** capability (and Background Modes → Remote notifications) to the App target.
3. `npx cap sync ios` (the `@capacitor/push-notifications` dep is already in package.json).
4. Deploy: `firebase deploy --only functions:expiryReminders`.
5. Verify: set a profile `medicalExpiry` 7 days out (Settings), confirm a token lands in `users/{uid}.fcmTokens`, then trigger the function (Cloud Scheduler "Run now" or the emulator) and confirm the push arrives.

One-time Prep Packs (non-consumables)

Code already present: webhook maps `pack_aip` / `pack_elpt` → `entitlement.packs[]` (in a transaction, preserving any Pro plan); `native-bridge.js purchasePack`; the paywall CTA routes `pack:*` features to it; `gate.js` / `entitlements.js hasPack` already unlock `packs/*.html`.

1. App Store Connect → create **non-consumable** products `pack_aip`, `pack_elpt` (extend `PACK_PRODUCTS` in `revenuecatWebhook.js` for more).
2. RevenueCat → add the products; the webhook handles `NON_RENEWING_PURCHASE`.
3. Verify: open `packs/aip.html` as a non-owner → paywall → "Unlock this pack" → sandbox purchase → webhook adds `aip` to `packs[]` → page unlocks; confirm a later Pro renewal does NOT drop the pack (transaction preserves both).

Phase 3 — web Stripe billing + B2B school licences (in the repo)

All entitlement writes now go through one transactional helper (`functions/entitlements.js`), shared by the Apple, Stripe and school paths, so a subscription, the one-time packs and a school seat never clobber each other.

Web Stripe (higher margin than Apple IAP)

A pilot who subscribes on the web gets the same cross-platform entitlement (Pro in the app too) without Apple's cut. **The app must never link to or mention web checkout (rule 3.1.1)** — `billing.js` refuses to start checkout in-app. Code present: `functions/stripe.js` (`createCheckoutSession` callable + `stripeWebhook`), `assets/js/billing.js` (`FGBilling.startProCheckout`).

1. Stripe Dashboard → create a **Product "Fly GACA Pro"** with two recurring **Prices**: SAR 59 / month and SAR 349 / year.
2. Secrets: `firebase functions:secrets:set STRIPE_SECRET_KEY STRIPE_WEBHOOK_SECRET STRIPE_PRICE_MONTHLY STRIPE_PRICE_ANNUAL`.
3. `cd functions && npm i && cd ..` (adds the `stripe` package), then `firebase deploy --only functions:createCheckoutSession,functions:stripeWebhook`.
4. Stripe → add a **webhook endpoint** at the deployed `stripeWebhook` URL, subscribing to `checkout.session.completed`, `customer.subscription.created/updated/deleted`. Put its

signing secret in `STRIPE_WEBHOOK_SECRET`.

5. **Activate the web CTA** (a launch task): on `pricing.html`, load `assets/js/billing.js` and wire the Pro CTA to `FGBilling.startProCheckout('annual'|'monthly')`. To make the web paywall actually block non-subscribers, set `WEB_GATING_LIVE = true` in `assets/js/gate.js` — do this only once the legal entity + checkout are live.
6. Verify: signed-in web user → checkout → test card → `checkout.session.completed` → `users/{uid}.entitlement` = Pro → sign in on iOS, Pro shows there too.

B2B school licences (sold off-app)

Schools buy seats on a contract; an operator then provisions cadets. Code present: `functions/school.js` (`grantSchoolLicence` / `revokeSchoolLicence`, admin-only).

1. Make an operator an admin (once): `admin.auth().setCustomUserClaims(uid, { admin: true })` (a small Node script with the service account, or the Firebase console function shell).
2. Deploy: `firebase deploy --only functions:grantSchoolLicence,functions:revokeSchoolLicence`.
3. Provision: call `grantSchoolLicence({ emails:[...], schoolId, expiresAt })` (e.g. from an admin tool or `firebase functions:shell`). Each cadet's entitlement becomes `{ plan:'school', source:'school', schoolId, expiresAt }`, which `isPro()` already treats as full access. Use `revokeSchoolLicence` at contract end.